

L6 Limbajul SQL

1. Funcții referitoare la o singură înregistrare

1. Obiective

1. SELECT folosind interfața grafică a aplicației Oracle SQL Developer
2. Funcții referitoare la o singură înregistrare (single-row functions):
 - A. Single-row functions;
 - B. Single-row functions - folosind interfața grafică a aplicației Oracle SQL Developer.

2. Considerații teoretice

1. SELECT folosind interfața grafică a aplicației Oracle SQL Developer

Pentru construirea unei comenzi SELECT folosind interfața grafică a aplicației *Oracle SQL Developer* se selectează succesiv **schema** (conexiunea) și tabul **Query Builder** – **figura 6.1**.

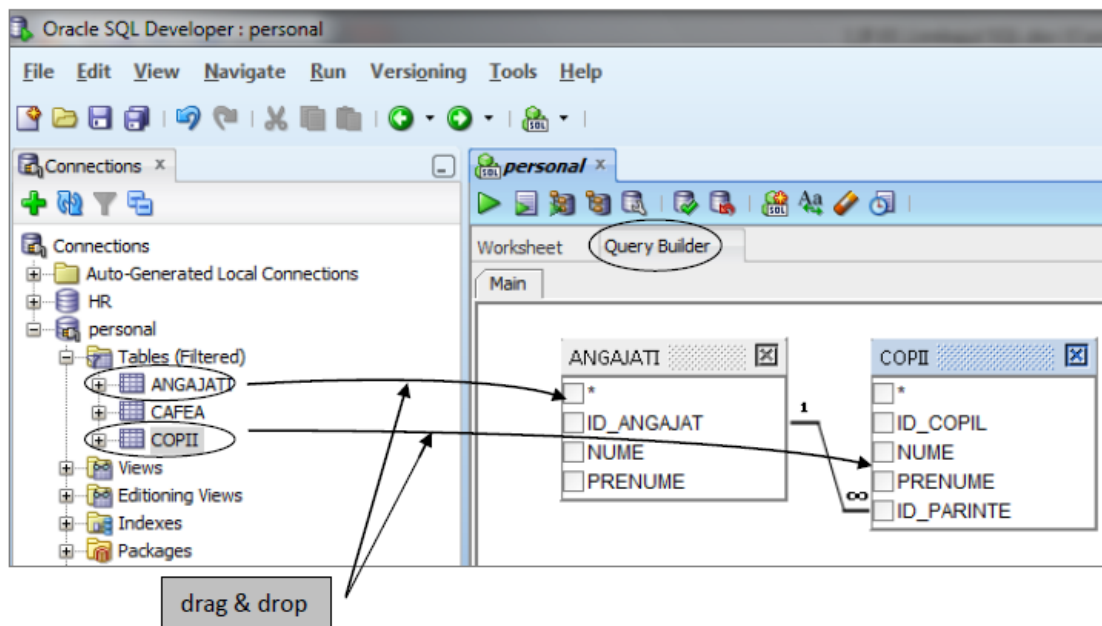


Figura 6.1. Activare SELECT folosind interfața grafică

După selectarea tabului Query Builder se aduc în zona grafică tabelele din care se selectează informații (târâire cu mouse-ul, *drag & drop*), ca în imagine.

Observație: Dacă între tabelele selectate există relații, ca în exemplul din figură (ANGAJATI.ID_ANGAJAT = COPII.ID_PARINTE), aplicația va simboliza aceasta printr-o linie. La capetele liniei apar simbolurile 1 și ∞ indicând o legătură de tip *1 la n*.

Câmpurile care trebuie să fie incluse în mulțimea de selecție se indică tot prin selectare cu mouse-ul, ca în figura 6.2:

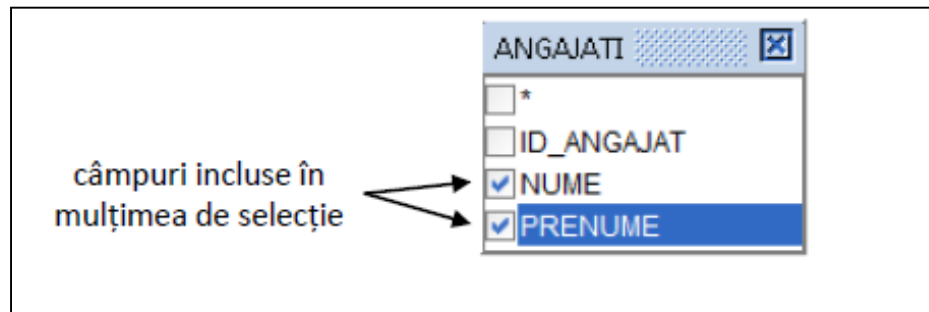


Figura 6.2. Selectare câmpuri pentru SELECT

2. Selectarea informațiilor dintr-un singur table (Funcții referitoare la o singură înregistrare (single-row functions))

Funcțiile sunt o caracteristică importantă a SQL și sunt utilizate pentru a realiza calcule asupra datelor, a modifica date, a manipula grupuri de înregistrări, a schimba formatul datelor sau pentru a converti diferite tipuri de date. Funcțiile se clasifică în:

- A) Funcții referitoare la o singură înregistrare;
- B) Funcții referitoare la mai multe înregistrări.

A)

Funcții referitoare la o singură înregistrare - sunt funcții utilizate pentru manipularea datelor individuale. Ele pot avea unul sau mai multe argumente și returnează o valoare pentru fiecare rând rezultat în urma interogării.

Funcțiile pe un singur rând cuprind următoarele tipuri de funcții:

1. funcții de tip caracter;
2. funcții de tip numeric;
3. funcții de tip data;
4. funcții de conversie;
5. funcții generale: NVL, NVL2, NULLIF, COALSECE, CASE, DECODE.

1) Funcții de tip caracter

Aceste funcții au ca argumente date de tip caracter și returnează date de tip VARCHAR2, CHAR sau NUMBER.

Cele mai importante funcții caracter sunt:

- CONCAT - returnează un șir de caractere format prin concatenarea a două șiruri;
- LOWER - modifică toate caracterele în litere mici;
- UPPER - modifică toate caracterele în litere mari;
- LENGTH - returnează numărul de caractere dintr-un anumit câmp;

- REPLACE - caută într-un șir de caractere un subșir iar dacă îl găsește îl va înlocui cu un alt șir de caractere;
- SUBSTR - returnează un subșir de caractere având o anumită lungime începând cu o anumită poziție;
- TRANSLATE - caută într-un prim șir de caractere fiecare dintre caracterele specificate într-un al 2-lea șir, caracterele găsite fiind înlocuite de cele specificate într-un al 3-lea șir.

Exemplu de utilizare a funcției LENGTH:

```
SQL> SELECT LENGTH (nume)
      FROM profesor;
```

Rezultat:

```
LENGTH(NUM)
-----
9
5
9
7
4
6
8
```

Spre deosebire de alte funcții, funcțiile caracter pot fi imbricate până la orice adâncime. Dacă funcțiile sunt imbricate ele sunt evaluate din interior spre exterior. Pentru a determina, de exemplu, de câte ori apare caracterul 'A' în câmpul nume vom folosi interogarea:

```
SQL> SELECT nume, LENGTH (nume)-LENGTH (TRANSLATE (nume,'DA','D'))
      FROM profesor;
```

Rezultat:

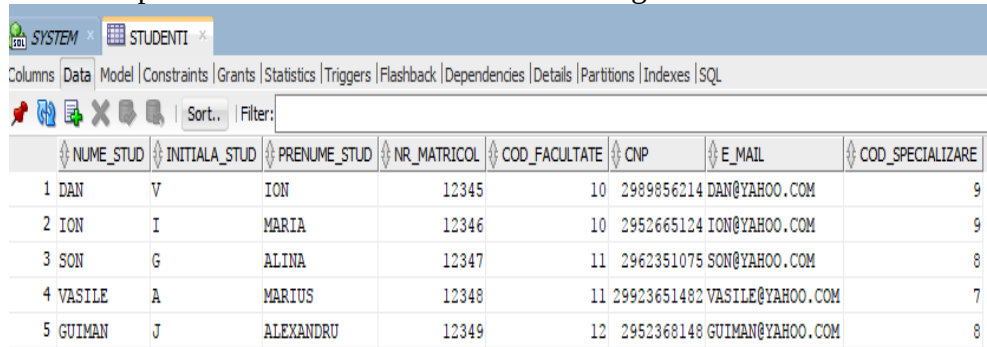
```
NUME      'A'
-----  ---
GHEORGHIU 0
MARIN      1
GEORGESCU 0
IONESCU    0
ALBU       1
VOINEA     1
STANESCU   1
```

Observații: în exemplul de mai sus, funcția TRANSLATE (nume, 'DA', 'D') va căuta în coloana "nume" primul caracter (caracterul 'D') din cel de-al doilea argument al funcției (șirul de caractere 'DA') și îl va înlocui cu primul caracter (adică tot cu caracterul 'D') din cel de-al treilea argument al funcției (șirul de caractere 'D'), apoi va căuta cel de-al doilea caracter, adică

caracterul 'A', și îl va șterge din câmpul nume deoarece acesta nu are caracter corespondent în cel de-al treilea argument al funcției. Am folosit acest artificiu deoarece șirul de caractere vid este echivalent cu valoarea Null, deci funcția TRANSLATE (nume, 'A', ' ') ar fi înlocuit toate valorile câmpului "nume" cu valoarea Null.

Exemple:

Pentru exempficare s-a utilibat baza de date din figura 6.3.:



	NUME_STUD	INITIALA_STUD	PRENUME_STUD	NR_MATRICOL	COD_FACULTATE	CNP	E_MAIL	COD_SPECIALIZARE
1	DAN	V	ION	12345	10	2989856214	DAN@YAHOO.COM	9
2	ION	I	MARIA	12346	10	2952665124	ION@YAHOO.COM	9
3	SON	G	ALINA	12347	11	2962351075	SON@YAHOO.COM	8
4	VASILE	A	MARIUS	12348	11	29923651482	VASILE@YAHOO.COM	7
5	GUIMAN	J	ALEXANDRU	12349	12	2952368148	GUIMAN@YAHOO.COM	8

Figura 6.3. Baza de date utilizată în exemple

1. CONCAT

SELECT CONCAT(*nume_student*, *prenume_student*)
FROM *lab6*; – figura 6.4.

```
CONCAT (NUME_STUDENT,PRENUME_STUDENT)
-----
DUMITRUVASILE
ANGHELIONUT
IORDACHEIULIANA
VASILESCUMIRUNA
ENESCUGEORGE
CRINVLAD
DINMARIAN
BRANALIN
SEBASTIANALEXANDRU
IANCURAMONA
```

Figura 6.4. Utilizare funcția character CONCAT

2. LOWER

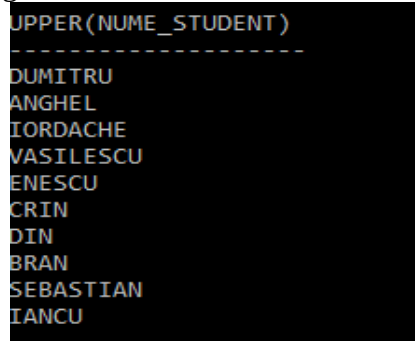
SELECT LOWER(*oras_student*)
FROM *lab6*; – figura 6.5.

```
LOWER(ORAS_STUDENT)
-----
pitesti
pitesti
c. de arges
craiova
bucuresti
timisoara
targu-jiu
pitesti
pitesti
cluj
```

Figura 6.5. Utilizare funcția character LOWER

3. UPPER

SELECT UPPER(*nume_student*)
FROM *lab6*; – figura 6.6.



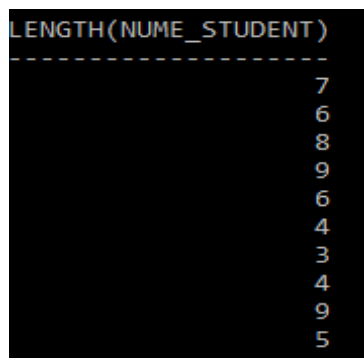
UPPER(NUME_STUDENT)

DUMITRU
ANGHEL
IORDACHE
VASILESCU
ENESCU
CRIN
DIN
BRAN
SEBASTIAN
IANCU

Figura 6.6. Utilizare funcția character UPPER

4. LENGTH

SELECT LENGTH(*nume_student*)
FROM *lab6*; – figura 6.7.



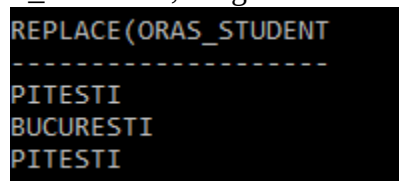
LENGTH(NUME_STUDENT)

7
6
8
9
6
4
3
4
9
5

Figura 6.7. Utilizare funcția character LENGTH

5. REPLACE

SELECT REPLACE(*oras_student*,*nume_student*)
FROM *lab6*
WHERE *an_studiu_student*=2; – figura 6.8.



REPLACE(ORAS_STUDENT

PITESTI
BUCURESTI
PITESTI

Figura 6.8. Utilizare funcția character REPLACE

6. SUBSTR

SELECT SUBSTR(*loc*,3)
FROM *student1*; – figura 6.9.

```
SQL> select substr(loc,3)
      2  from student1;

SUBSTR(LOC,3)
-----
C
ngu-Jiu
ngu-Jiu
C
ngu-Jiu
SQL>
```

Figura 6.9. Utilizare funcția character SUBSTR

7. TRANSLATE

SELECT nume_student, LENGTH(nume_student)-LENGTH(TRANSLATE(nume_student,'DA','D'))
FROM lab6; – figura 6.10.

NUME_STUDENT	'A'
ANGHEL	1
DUMITRU	0
VASILESCU	1
SEBASTIAN	2
BRAN	1
ENESCU	0
CRIN	0
DIN	0
IANCU	1
IORDACHE	1

Figura 6.10. Utilizare funcția character TRANSLATE

2) Funcții de tip numeric sau aritmetic

Aceste funcții au ca argumente date numerice și returnează tot valori numerice. Marea majoritate a acestor funcții au o precizie de 38 de zecimale (COS, EXP, LN, LOG, SIN, SQRT, TAN au însă o precizie de 36 de zecimale).

Cele mai importante funcții sunt:

- ROUND - rotunțește valorile la un anumit număr de poziții zecimale;
- TRUNC - trunchiază valorile la un anumit număr de poziții zecimale;
- CEIL - returnează cel mai mic întreg mai mare sau egal cu o anumită valoare;
- FLOOR - returnează cel mai mare întreg mai mic sau egal cu o anumită valoare;
- SIGN - returnează valoarea -1 dacă valoarea argumentului primit este mai mică decât 0, returnează valoarea 1 dacă valoarea argumentului primit este mai mare decât 0 și 0 dacă valoarea argumentului primit este egală cu 0;

SQRT - returnează rădăcina pătrată a argumentului primit;
 ABS - returnează valoarea absolută a argumentului primit;
 POWER - returnează valoarea unui număr ridicat la o anumită putere;
 MOD - returnează restul împărțirii a două numere;
 LOG, SIN, alte funcții matematice
 TAN, COS,
 EXP, LN

Exemplu Sintaxă:

Funcție	Descriere
ROUND(column expression, n)	rotunjește la o valoare cu n zecimale
TRUNC(column expression, n)	se obține o valoare cu n zecimale prin trunchiere
MOD(m, n)	returnează restul împărțirii lui m la n

Exemple:

```

SELECT nume, sala, MOD (sala,100)
FROM salariati
WHERE funct = 'VÂNZATOR'

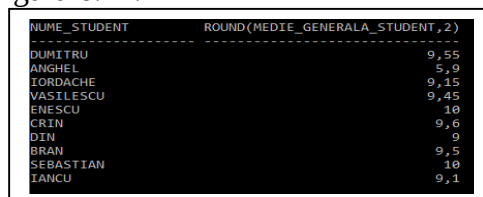
```

1. ROUND

```

SELECT ROUND(medie_generala_student,2)
FROM lab6; figura 6.11.

```



NUME_STUDENT	ROUND(MEDIE_GENERALA_STUDENT,2)
DUMITRU	9,55
ANGHEL	5,9
IORDACHE	9,15
VASILESCU	9,45
ENESCU	10
CRIN	9,6
DIN	9
BRAN	9,5
SEBASTIAN	10
IANCU	9,1

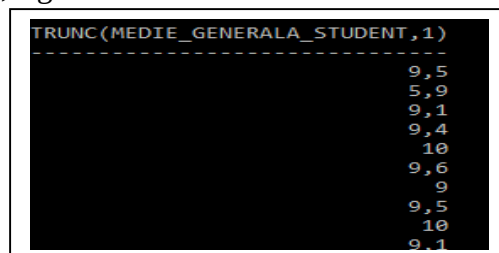
Figura 6.11. Utilizare funcția numeric sau aritmetic ROUND

2. TRUNC

```

SELECT TRUNC(medie_generala_student,1)
FROM lab6; figura 6.12.

```



TRUNC(MEDIE_GENERALA_STUDENT,1)
9,5
5,9
9,1
9,4
10
9,6
9
9,5
10
9,1

Figura 6.12. Utilizare funcția numeric sau aritmetic TRUNC

3. CEIL

```

SELECT CEIL(medie_generala_student)

```

FROM lab6; figura 6.13.

CEIL(MEDIE_GENERALA_STUDENT)
10
6
10
10
10
10
9
10
10
10

Figura 6.13. Utilizare funcția numeric sau arithmetic CEIL

4. FLOOR figura 6.14.

FLOOR(PUNCTAJ)
2345
1300
3000
1234
3456

Figura 6.14. Utilizare funcția numeric sau arithmetic FLOOR

5. SIGN

SELECT nume_student,an_studiu_student,SIGN(medie_generala_student) FROM lab6;
figura 6.15.

NUME_STUDENT	AN_STUDIU_STUDENT	SIGN(MEDIE_GENERALA_STUDENT)
DUMITRU	1	1
ANGHEL	2	1
IORDACHE	3	1
VASILESCU	1	1
ENESCU	2	1
CRIN	3	1
DIN	4	1
BRAN	1	1
SEBASTIAN	2	1
IANCU	3	1

Figura 6.15. Utilizare funcția numeric sau arithmetic SIGN

6. SQRT

SELECT nume_student,oras_student,SQRT(medie_generala_student)
FROM lab6; figura 6.16.

NUME_STUDENT	ORAS_STUDENT	SQRT(MEDIE_GENERALA_STUDENT)
DUMITRU	PITESTI	3,09030743
IORDACHE	C. DE ARGES	3,02489669
VASILESCU	CRATOVA	3,07408523
ENESCU	BUCURESTI	3,16227766
CRIN	TIMISOARA	3,09838668
DIN	TARGU-JIU	3
BRAN	PITESTI	3,082207
SEBASTIAN	PITESTI	3,16227766
IANCU	CLUJ	3,01662063

Figura 6.16. Utilizare funcția numeric sau arithmetic SQRT

7. ABS

SELECT *nume_student,an_studiu_student,ABS(an_studiu_student)* FROM *lab6*; figura 6.17.

```
SQL> SELECT NUME_STUDENT,AN_STUDIU_STUDENT,ABS(AN_STUDIU_STUDENT)
2 FROM lab6;
```

NUME_STUDENT	AN_STUDIU_STUDENT	ABS(AN_STUDIU_STUDENT)
DUMITRU	1	1
ANGHEL	2	2
TORDACHE	3	3
VASILESCU	1	1
ENESCU	2	2
CRIN	3	3
DIN	4	4
BRAN	1	1
SEBASTIAN	2	2
TANCU	3	3

Figura 6.17. Utilizare funcția numeric sau arithmetic ABS

8. POWER

SELECT *nume_student,an_studiu_student,POWER(an_studiu_student,2)* FROM *lab6*; figura 6.18.

NUME_STUDENT	AN_STUDIU_STUDENT	POWER(AN_STUDIU_STUDENT,2)
DUMITRU	1	1
ANGHEL	2	4
TORDACHE	3	9
VASILESCU	1	1
ENESCU	2	4
CRIN	3	9
DIN	4	16
BRAN	1	1
SEBASTIAN	2	4
TANCU	3	9

Figura 6.18. Utilizare funcția numeric sau arithmetic POWER

9. MOD

SELECT *MOD(punctaj,tutor)*
FROM *student1*; figura 6.19.

```
SQL> select mod(punctaj,tutor)
2 from student1;
```

MOD(PUNCTAJ,TUTOR)
343
298
998
232
447

Figura 6.19. Utilizare funcția numeric sau arithmetic MOD

10. LOG, SIN, TAN, COS, EXP, LN

SELECT *EXP(an_studiu_student),SIN(medie_generala_student),COS(an_studiu_student)*
FROM *lab6*; figura 6.20.

EXP(AN_STUDIU_STUDENT)	SIN(MEDIE_GENERALA_STUDENT)	COS(AN_STUDIU_STUDENT)
2,71828183	-,12489504	,540302306
7,3890561	-,37387666	-,41614684
20,0855369	-,271333234	-,9899925
2,71828183	-,02521937	,540302306
7,3890561	-,54402111	-,41614684
20,0855369	-,17432678	-,9899925
54,59815	,412118485	-,65364362
2,71828183	-,07515112	,540302306
7,3890561	-,54402111	-,41614684
20,0855369	,319098362	-,9899925

Figura 6.20. Utilizare funcția numeric sau arithmetic SIN, COS, EXP

3) Funcții de tip dată calendaristică și oră

În Oracle datele de tip dată calendaristică sunt reprezentate sub un format numeric reprezenând: ziua, luna, anul, ora, minutul, secunda și secolul. Oracle poate manevra date calendaristice de la 1 ianuarie 4712 î. Cr până la 31 decembrie 4712 d. Cr. Modul implicit de afișare și introducere este sub forma: DD-MON-W (ex. '31-Dec-99'). Această categorie de funcții operează pe valori de tip dată calendaristică, rezultatul returnat fiind tot de tip dată calendaristică, excepție făcând funcția MONTHS BETWEEN care returnează o valoare numerică.

Asupra datelor calendaristice se pot realiza operații aritmetice, cum ar fi scăderea sau adunarea, modul lor de funcționare fiind ilustrat în tabelul 6.1. de mai jos:

Tabelul 6.1. Operații aritmetice cu date calendaristice

Tip operand	Operatie	Tip operand	Tip rezultat	Descriere
data	+/-	numar	data	Adauga/scade un numar de zile la o data calendaristica
data	+/-	numar/24	data	Adauga/scade un numar de ore la o data calendaristica
data	+/-	numar/ 1440	data	Adauga/scade un numar de minute la o data calendaristica
data	+/-	numar/86400	data	Adauga/scade un numar de secunde la o data calendaristica

Tip operand	Operatie	Tip operand	Tip rezultat	Descriere
data	-	data	numar zile	Scade doua date calendaristice rezultand diferenta in numar de zile. Daca al doilea operand este mai mare decat primul numarul de zile rezultat este reprezentat de o valoare negativa

De asemenea, mai există funcțiile ROUND și TRUNC care rotunjesc, respectiv trunchiază data calendaristică. Aceste funcții sunt foarte folositoare atunci când se dorește compararea datelor calendaristice care au ora diferită. Exemplul următor rotunjește data de naștere a cadrelor didactice în funcție de an:

```
SQL> SELECT ROUND(data nast,'year') 'data'
      FROM profesor;
```

Resultat:

DATA

— — — —

01-JAN-47

01-JAN-45

01-JAN 52

01-JAN-66

01-JAN-70

Exemplu:

```
SQL> Select NUME_STUDENT,PRENUME_STUDENT,DATA_NASTERE_STUDENT
2 FROM LAB6;
```

NUME_STUDENT	PRENUME_STUDENT	DATA_NASTE
DUMITRU	VASILE	20-04-1995
ANGHEL	IONUT	10-02-1995
IORDACHE	IULIANA	11-09-1992
VASILESCU	MIRUNA	10-03-1997
ENESCU	GEORGE	01-09-1996
CRIN	VLAD	30-10-1990
DIN	MARIAN	05-01-1980
BRAN	ALIN	12-01-1998
SEBASTIAN	ALEXANDRU	06-09-1997
IANCU	RAMONA	11-12-1995

```
SELECT ROUND(data_nastere_student, 'year')
```

FROM *lab6*;

Ergebnis:

```

DATA
-----
01-01-1995
01-01-1995
01-01-1993
01-01-1997
01-01-1997
01-01-1991
01-01-1980
01-01-1998
01-01-1998
01-01-1996

```

Funcția **SYSDATE** - întoarce ca rezultat data calendaristică a sistemului

Sintaxa:

SELECT SYSDATE

FROM DUAL

Efect: returnează data și ora sistemului.

SYSDATE se folosește ca orice alt nume de coloană, se poate afișa data curentă selectând SYSDATE dintr-o tabelă.

Exemplu:

SELECT SYSDATE

FROM *lab6*;

Ergebnis:

[illegible]

Deoarece o bază de date conține atât numere, cât și date, se pot folosi operatorii aritmetici ca adunarea și scăderea între numere și date.

Operatie	Rezultat	Descriere
data + numar	Data	Adauga un numar de zile la o anumita data
data - numar	Data	Scade un numar de zile dintr-o anumita data
data - data	Numar de zile	Scade o data din alta data
data + number/24	Data	Adauga un numar de ore la o data

Principalele funcții de tip dată folosite în Oracle sunt:

1. ***MONTHS_BETWEEN (date1, date2):***

Returnează numărul de luni dintre *date1* și *date2*. Rezultatul poate fi pozitiv sau negativ. Dacă *date1* este mai mare decât *date2* rezultatul este pozitiv, dacă *date1* este mai mică decât *date2* rezultatul este negativ. Valoarea zecimală reprezintă o parte a lunii.

2. ***ADD_MONTHS (date, n):***

Adaugă *n* luni la o *date*. *n* trebuie să aibă o valoare întreagă și poate fi negativ.

Exemplu 1:

```
SELECT  nume_student,prenume_student,ADD_MONTHS(data_nastere_student,3)  FROM
lab6;
```

Rezultat:

NUME_STUDENT	PRENUME_STUDENT	ADD_MONTHS
DUMITRU	VASILE	20-07-1995
ANGHEL	IONUT	10-05-1995
IORDACHE	IULIANA	11-12-1992
VASILESCU	MIRUNA	10-06-1997
ENESCU	GEORGE	01-12-1996
CRIN	VLAD	30-01-1991
DIN	MARIAN	05-04-1980
BRAN	ALIN	12-04-1998
SEBASTIAN	ALEXANDRU	06-12-1997
IANCU	RAMONA	11-03-1996

```
SELECT ADD_MONTHS (datan,3)
FROM student;
```

Rezultat:

```
SQL> select ADD_MONTHS (datan,3)
2  from student;

ADD_MONTHS
-----
12-06-1987
02-07-1989
21-06-1968
05-08-1989
06-12-1990
```

3. ***NEXT_DAY (date, 'char'):***

Returnează data următoare unei zile specificate. Argumentul *char* poate fi un număr sau un șir de caractere.

Exemplu 1:

```
SELECT nume_student,prenume_student,NEXT_DAY(data_nastere_student,'luni') FROM lab6;
```

Rezultat:

NUME_STUDENT	PRENUME_STUDENT	NEXT_DAY(D
DUMITRU	VASILE	24-04-1995
ANGHEL	IONUT	13-02-1995
IORDACHE	IULIANA	14-09-1992
VASILESCU	MIRUNA	17-03-1997
ENESCU	GEORGE	02-09-1996
CRIN	VLAD	05-11-1990
DIN	MARIAN	07-01-1980
BRAN	ALIN	19-01-1998
SEBASTIAN	ALEXANDRU	08-09-1997
IANCU	RAMONA	18-12-1995

Exemplu 2:

```
SELECT NEXT_DAY(datan,'luni')  
FROM student;
```

Rezultat:

```
SQL> select NEXT_DAY(datan,'luni')  
2 from student;  
  
NEXT_DAY(D  
-----  
16-03-1987  
03-04-1989  
25-03-1968  
08-05-1989  
10-09-1990
```

4. **LAST_DAY (date):**

Returnează ultima zi din lună.

Exemplu 1:

```
SELECT nume_student,prenume_student,LAST_DAY(data_nastere_student) FROM lab6;
```

Rezultat:

NUME_STUDENT	PRENUME_STUDENT	LAST_DAY(D
DUMITRU	VASILE	30-04-1995
ANGHEL	IONUT	28-02-1995
IORDACHE	IULIANA	30-09-1992
VASILESCU	MIRUNA	31-03-1997
ENESCU	GEORGE	30-09-1996
CRIN	VLAD	31-10-1990
DIN	MARIAN	31-01-1980
BRAN	ALIN	31-01-1998
SEBASTIAN	ALEXANDRU	30-09-1997
IANCU	RAMONA	31-12-1995

Exemplu 2:

```
SELECT LAST_DAY(datan)  
FROM student;
```

Rezultat:

```
SQL> select last_day(datan)  
2 from student;  
  
LAST_DAY(D  
-----  
31-03-1987  
30-04-1989  
31-03-1968  
31-05-1989  
30-09-1990
```

5. **ROUND (date [, 'fmt']):**

Returnează data rotunjită la unitatea specificată de *fmt*. Dacă formatul *fmt* lipsește data este rotunjită la cea mai apropiată zi.

Exemplu:

```
SELECT ROUND(datan,'year')
FROM student;
```

Rezultat:

```
SQL> select round(datan,'year')
      2  from student
      3  ;

ROUND(DATA
-----
01-01-1987
01-01-1989
01-01-1968
01-01-1989
01-01-1991
```

6. **TRUNC (date [, 'fmt']):**

Returnează data truncată la unitatea specificată de *fmt*. Dacă formatul *fmt* lipsește data este rotunjită la cea mai apropiată zi.

Exemplu 1:

NUME_STUDENT	PRENUME_STUDENT	TRUNC(DATA)
DUMITRU	VASILE	20-04-1995
ANGHEL	IONUT	10-02-1995
IORDACHE	IULIANA	11-09-1992
VASILESCU	MIRUNA	10-03-1997
ENESCU	GEORGE	01-09-1996
CRIN	VLAD	30-10-1990
DIN	MARIAN	05-01-1980
BRAN	ALIN	12-01-1998
SEBASTIAN	ALEXANDRU	06-09-1997
IANCU	RAMONA	11-12-1995

Exemplu 2:

```
SELECT TRUNC(datan,'year')
FROM student;
```

Rezultat:

```
SQL> select trunc(datan,'year')
      2  from student;

TRUNC(DATA
-----
01-01-1987
01-01-1989
01-01-1968
01-01-1989
01-01-1990
```

Exemple:

```
SELECT nume, data_ang
FROM angajati
WHERE nume like '%S'
```

```
SELECT nume, (SYSDATE - data_ang)/7 AS SAPTAMANI
FROM angajati
WHERE job = 'VANZATOR'
```

```
SELECT nr_ang, data_ang, ROUND(data_ang, 'MONTH'), TRUNC(data_ang,'MONTH')
FROM angajati
WHERE data_ang LIKE '%80'
```

4) Funcții de conversie

În unele cazuri Oracle folosește date de un anumit tip atunci când așteaptă date de un alt tip. Când se întâmplă acest lucru, Oracle poate converti automat datele în tipul de date dorit.

De *exemplu*, nu se poate înmulți 3 cu 7 și apoi aduna '10N'. Prin urmare se realizează anumite conversii.

Această conversie poate fi făcută *implicit* de serverul Oracle sau *explicit* de utilizator. Conversia *implicită* se face conform anumitor reguli, iar conversia *explicită* se face folosind funcții de conversie.

Conversiile implicite se realizează în următoarele cazuri:

- **atribuiri de valori** unei coloane (folosind comenzile INSERT sau UPDATE) sau atribuirilor de valori unor argumente ale unei funcții;
- **evaluări de expresii**.

Pentru **atribuiri**, programul Oracle efectuează în mod implicit următoarele conversii de tip:

- VARCHAR2 sau CHAR la NUMBER
- VARCHAR2 sau CHAR la DATE
- VARCHAR2 sau CHAR la ROWID
- NUMBER, DATE sau ROWID la VARCHAR2

Conversia la atribuire reușește în cazul în care Oracle poate converti tipul valorii atribuite la tipul destinației atribuirii.

Pentru **evaluarea expresiilor**, se realizează în mod implicit următoarele conversii de tip:

- VARCHAR2 sau CHAR la NUMBER
- VARCHAR2 sau CHAR la DATE
- VARCHAR2 sau CHAR la ROWID

De *exemplu*, pentru următoarea interogare se realizează conversia în mod implicit a constantei de tip CHAR, '10', la tipul NUMBER.

```
SQL> SELECT salariu + '10'  
      FROM profesor;
```

Rezultat:

```
SALARIU+'10'  
-----  
3010  
2510  
2810  
1510  
2210  
1210  
1210
```

Exemplu:

NUME_STUDENT	PRENUME_STUDENT	MEDIE_GENERALA_STUDENT+'0,10'
DUMITRU	VASILE	9,65
ANGHEL	IONUT	6
IORDACHE	IULIANA	9,25
VASILESCU	MIRUNA	9,55
ENESCU	GEORGE	10,1
CRIN	VLAD	9,7
DIN	MARIAN	9,1
BRAN	ALIN	9,6
SEBASTIAN	ALEXANDRU	10,1
TANCU	RAMONA	9,2

Pentru **conversiile explicite** de tip, SQL pune la dispoziție mai multe funcții de conversie, de la un anumit tip de dată la altul, după cum este arătat în tabelul de mai jos.

Tip convertit	CHAR	NUMBER	DATE	RAW	ROWID
Tip initial					
CHAR		TO_NUMBER	TO_DATE	HEXTORAW	CHARTOROWID
NUMBER	TO_CHAR		TO_DATE(nr,'J')		
DATE	TO_CHAR	TO_DATE(data,'J')			
RAW	RAWTOHEX	-			
RAWID	RAWIDTOCHAR				

Cele mai uzuale funcții sunt:

Funcție	Descriere
TO_CHAR (number date, [fmt])	Converteste o valoare de tip numeric sau data într-un sir de caractere ce are formatul specificat optional de <i>fmt</i> .
TO_NUMBER (number date, [fmt])	Converteste un sir de caractere ce contine cifre într-un numar in formatul specificat optional de <i>fmt</i> .
TO_DATE (number date, [fmt])	Converteste un sir de caractere reprezentand o data într-o valoare de tip data avand formatul <i>fmt</i> specificat. Daca formatul nu este specificat, formatul este DD-MON-YY.

Pentru a realiza conversia, aceste funcții folosesc anumite măști de format.

Exemple:

TO_CHAR

1. Următorul exemplu va prelua data și ora curentă a sistemului din funcția SYSDATE și o va formata într-o dată scrisă pe litere ce va conține și ora în minute și secunde:

```
SQL> SELECT TO_CHAR (SYSDATE,'DD MONTH YYYY HH24: MI: SS') data
      FROM dual;
```

Rezultat:

```
DATA
-----
17 MAY      2000 17:03:38
```

2.

```
SELECT nr_ang, TO_CHAR (data_ang, 'MM/YY') Luna_Angajarii
FROM angajati
WHERE nume = 'POPA'
```


3.

```
SELECT nume, TO_CHAR
(data_ang, 'fmDD Month YYYY') AS data_angajarii FROM angajati
```

4.

```
SELECT TO_CHAR (salariu, '$99,999.00') SALARIU
FROM angajati
WHERE nume = 'FORD'
```

5. **SELECT** nume_student, prenume_student, TO_CHAR(data_nastere_student, 'mm/yy') FROM lab6;

Rezultat:

NUME_STUDENT	PRENUME_STUDENT	TO_CH
DUMITRU	VASILE	04/95
ANGHEL	IONUT	02/95
IORDACHE	IULIANA	09/92
VASILESCU	MIRUNA	03/97
ENESCU	GEORGE	09/96
CRIN	VLAD	10/90
DIN	MARIAN	01/80
BRAN	ALIN	01/98
SEBASTIAN	ALEXANDRU	09/97
IANCU	RAMONA	12/95

5. TO_NUMBER

SELECT nume_student, prenume_student, TO_NUMBER(medie_generala_student) FROM lab6;

Rezultat:

NUME_STUDENT	PRENUME_STUDENT	TO_NUMBER(MEDIE_GENERALA_STUDENT)
DUMITRU	VASILE	9,55
ANGHEL	IONUT	5,9
IORDACHE	IULIANA	9,15
VASILESCU	MIRUNA	9,45
ENESCU	GEORGE	10
CRIN	VLAD	9,6
DIN	MARIAN	9
BRAN	ALIN	9,5
SEBASTIAN	ALEXANDRU	10
IANCU	RAMONA	9,1

5) Funcții generale

Funcțiile generale sunt: **NVL**, **NVL2**, **NULLIF** și **COALESCE**, aceste funcții lucrează cu orice tip de date.

Funcție	Descriere
NVL (expr1, expr2)	Converteste o valoare nula într-o valoare actuală.
NVL2 (expr1, expr2, expr3)	Dacă <i>expr1</i> nu este nula, NVL2 returnează <i>expr2</i> . Dacă <i>expr1</i> este nula, NVL2 returnează <i>expr3</i> . Argumentul <i>expr1</i> poate fi orice tip de date.
NULLIF (expr1, expr2)	Compara două expresii și returnează valoarea nula dacă sunt egale sau prima expresie dacă nu sunt egale.
COALESCE (expr1, expr2, ..., expr n)	Returnează prima expresie non-nula din lista de expresii.

Exemple:

- Funcția **NVL** convertește o valoare nulă într-o valoare actuală. Funcția NVL se poate folosi pentru a converti orice tip de date, dar valoarea rezultată are întotdeauna același tip cu *expr1*.

```
SELECT nume, salariu, NVL (comision, 0), (salariu*12) + (salariu*12*NVL (comision, 0))
AN_SAL
FROM angajati
```

```
SELECT nume, salariu, comision, (salariu*12) + (salariu*12*comision)
AN_SAL FROM emp
```

Exemplu:

❖	COLUMN_NAME	❖	DATA_TYPE	❖	NULLABLE	DATA_DEFAULT	❖	COLUMN_ID	❖	COMMENTS
1	NUME_ANGAJATI		VARCHAR2(20 BYTE)	No		(null)		1	(null)	
2	PRENUME_ANGAJATI		VARCHAR2(20 BYTE)	No		(null)		2	(null)	
3	ID_ANGAJAT		NUMBER(5,0)	No		(null)		3	(null)	
4	FUNCTIE_ANGAJAT		VARCHAR2(20 BYTE)	No		(null)		4	(null)	
5	SALARIU_ANGAJAT		NUMBER(20,0)	No		(null)		5	(null)	
6	COMISION_ANGAJAT		NUMBER(20,0)	No		(null)		6	(null)	

1	Marinescu	Ionut	123	DIRECTOR	50000	100
2	Dorin	Amalia	124	HR	25000	50
3	Nistor	Vasile	125	INGINER	35000	25
4	Miron	Alina	126	ECONOMIST	30000	25
5	Doru	Cristian	127	PROGRAMATOR	35000	10
6	Balint	Marin	128	PAZNIC	20000	0

```
SELECT      nume_angajati,prenume_angajati,NVL(comision_angajat,0),(salariu_angajat*2)
FROM lab6;
```

NUME_ANGAJATI	PRENUME_ANGAJATI	NVL(COMISION_ANGAJAT,0)

(SALARIU_ANGAJAT*2)		

Marinescu	Ionut	100
100000		
Dorin	Amalia	50
50000		
Nistor	Vasile	25
70000		

NUME_ANGAJATI	PRENUME_ANGAJATI	NVL(COMISION_ANGAJAT,0)

(SALARIU_ANGAJAT*2)		

Miron	Alina	25
60000		
Doru	Cristian	10
70000		
Balint	Marin	0
40000		

- b) Functia **NVL2** examinează prima expresie. Dacă prima expresie nu este nulă, atunci funcția **NVL2** returnează a doua expresie. Dacă prima expresie este nulă, atunci a treia expresie este returnată.

```
SELECT nume, salariu, comision, NVL2 (comision, 'sal+comm', 'sal') INCOME
FROM angajati
WHERE nr_dept = 30
```

Exemplu:

```
SELECT nume,an,tutor,NVL2(tutor,'an+tutor','an') income
FROM student;
```

```
SQL> select nume,an,tutor,nvl2(tutor,'an+tutor','an') income
2 from student;
```

NUME	AN	TUTOR
Popa Marcel	1	1001
Popescu Ion	4	1002
Avram Nicolae	1	1004

NUME	AN	TUTOR
Ionescu Mariana	4	1002
Popescu Gina	3	1003

- c) Funcția **NULLIF** compară două expresii. Dacă cele două expresii sunt egale, funcția returnează valoare nulă. Dacă cele două expresii nu sunt egale, funcția returnează prima expresie. Funcția **NULLIF** este echivalentă cu expresia **CASE**.

```
SELECT nume, LENGTH (nume) EXPR_1, functie, LENGTH (functie) EXPR_2,
NULLIF (LENGTH (nume), LENGTH (functie)) RESULT FROM angajati
```

- d) Funcția **COALESCE** returnează prima expresie care nu este nulă.

```
SELECT nume, COALESCE (comision, salariu, 10) COMM
FROM angajati
ORDER BY commission
```

Exemplu:

```
SELECT nume_angajati,prenume_angajati,
COALESCE(comision_angajat,salariu_angajat,10)COMM FROM lab6 ORDER BY
comision_angajat;
```

NUME_ANGAJATI	PRENUME_ANGAJATI	COMM
Balint	Marin	0
Doru	Cristian	10
Nistor	Vasile	25
Miron	Alina	25
Dorin	Amalia	50
Marinescu	Ionut	100

6) Funcții diverse

Acestea sunt în general funcții care acceptă ca argumente orice tip de dată. Cele mai utilizate sunt:

- **DECODE** - Aceasta este una dintre cele mai puternice funcții SQL. Practic, aceasta facilitează interogările condiționate, acționând ca o comandă 'if-then-else' sau 'case' dintr-un

limbaj procedural. Pentru fiecare înregistrare se va evalua valoarea din coloana testată și se va compara pe rând cu fiecare valoare declarată în cadrul funcției. Dacă se găsesc valori egale, atunci funcția va returna o valoare aferentă acestei egalități, declarată tot în cadrul funcției. Se poate specifica ca, în cazul în care nu se găsesc valori egale, funcția să întoarcă o anumită valoare. Dacă acest lucru nu se specifică funcția va întoarce valoarea Null.

- **GREATEST** - returnează cea mai mare valoare dintr-o listă de valori;
- **LEAST** - returnează cea mai mică valoare dintr-o listă de valori;
- **VSIZE** - returnează numărul de bytes pe care este reprezentată intern o anumită coloană;
- **USER** - returnează numele utilizatorului curent al bazei de date;
- **DUMP** - returnează o valoare ce conține codul tipului de dată, lungimea în bytes, precum și reprezentarea internă a unei expresii.

Exemplul 1 următor utilizează funcția **DECODE** pentru a returna o creștere a salariului cadrelor didactice cu grad de profesor, conferențiar și lector, restul salariilor rămânând nemodificate:

```
SQL> SELECT nume, grad, salariu,
        DECODE (grad,'PROF',salariu*1.2, 'CONF',salariu*1.15,
        'LECT', salariu*1.1, salariu) 'Salariu modificat'
FROM profesor;
```

NUME	GRAD	SALARIU	Salariu modificat
----	----	-----	-----
GHEORGHIU	PROF	3000	3600
MARIN	PROF	2500	3000
GEORGESCU	CONF	2800	3220
IONESCU	ASIST	3500	1500
ALBU	LECT	2200	2420
VOINEA	ASIST	1200	1200
STANESCU	ASIST	1200	1200

Exemplul 2:

```
SELECT nume_angajati,prenume_angajati,
DECODE(funcție_angajat,'HR',salariu_angajat*1,5,'INGINER',salariu_angajat*1.5) FROM
lab6;
```

NUME_ANGAJATI	PRENUME_ANGAJATI	
DECODE(FUNCTIE_ANGAJAT, 'HR', SALARIU_ANGAJAT*1,5, 'INGINER', SALARIU_ANGAJAT*1.5)		
Marinescu	Ionut	75000
Dorin	Amalia	25000
Nistor	Vasile	52500
NUME_ANGAJATI PRENUME_ANGAJATI		
DECODE(FUNCTIE_ANGAJAT, 'HR', SALARIU_ANGAJAT*1,5, 'INGINER', SALARIU_ANGAJAT*1.5)		
Hiron	Alina	45000
Doru	Cristian	52500
Balint	Marin	30000

Exemplul 3:

```
SELECT nume_angajati, prenume_angajati, functie_angajat, GREATEST(salariu_angajat)
FROM lab6;
```

NUME_ANGAJATI	PRENUME_ANGAJATI	FUNCTIE_ANGAJAT
GREATEST(SALARIU_ANGAJAT)		
Marinescu	Ionut	DIRECTOR
Dorin	Amalia	HR
Nistor	Vasile	INGINER
Miron	Alina	ECONOMIST
Doru	Cristian	PROGRAMATOR
Balint	Marin	PAZNIC

Exemplul 4:

```
SELECT nume_angajati, prenume_angajati, LEAST(salariu_angajat) FROM lab6 ;
```

NUME_ANGAJATI	PRENUME_ANGAJATI	LEAST(SALARIU_ANGAJAT)
Marinescu	Ionut	50000
Dorin	Amalia	25000
Nistor	Vasile	35000
Miron	Alina	30000
Doru	Cristian	35000
Balint	Marin	20000

Exemplul 5:

```
SELECT VSIZE(nume)
FROM student;
```

```
SQL> select vsize(nume)
      2 from student;

VSIZE(NUM)
-----
      11
      11
      13
      15
      12
```

Exemplul 6:

```
SELECT DUMP(salariu_angajat)
FROM lab6;
```

```
DUMP(SALARIU_ANGAJAT)
-----
Typ=2 Len=2: 195,6
Typ=2 Len=3: 195,3,51
Typ=2 Len=3: 195,4,51
Typ=2 Len=2: 195,4
Typ=2 Len=3: 195,4,51
Typ=2 Len=2: 195,3
```

Exemplul 7:

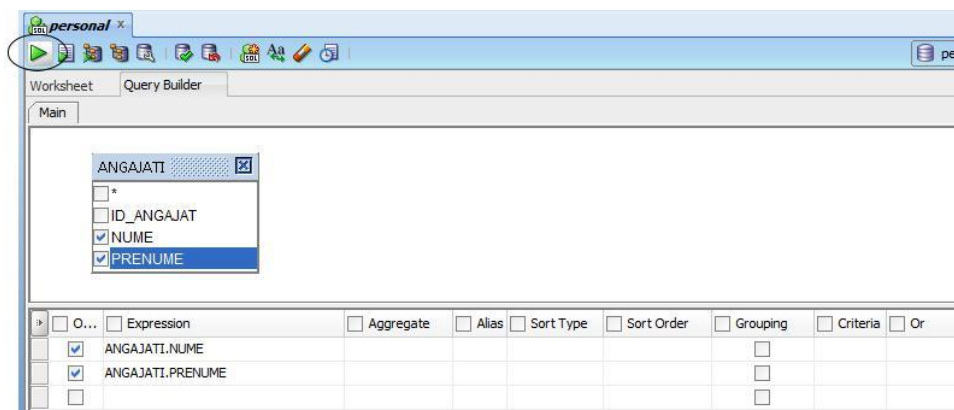
```
SELECT USER
FROM lab6;
```

```
USER
-----
OUTLN
OUTLN
OUTLN
OUTLN
OUTLN
OUTLN
```

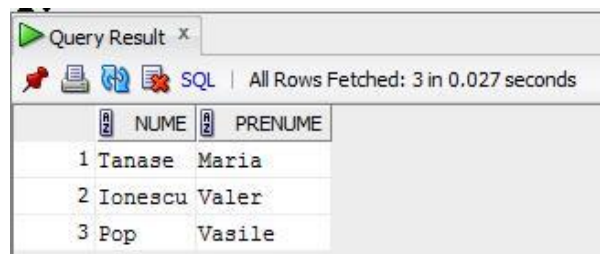
B) Folosind interfața grafică a aplicației Oracle SQL Developer

Exemple fundamentale:

- **SELECT** *Nume, Prenume* **FROM** *angajati*

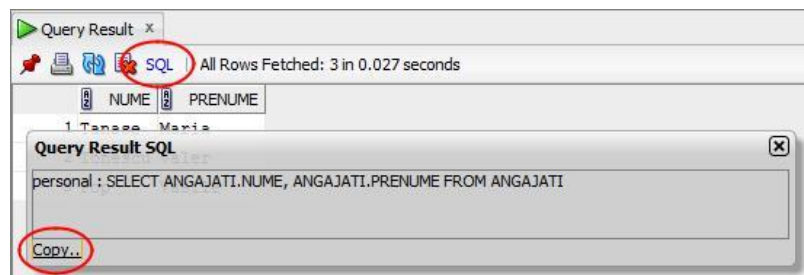


După precizarea câmpurilor se apasă butonul  (*Run Statement*) și aplicația afișează rezultatul executării comenzii **SELECT** create.



	NUME	PRENUME
1	Tanase	Maria
2	Ionescu	Valer
3	Pop	Vasile

Pentru a vedea comanda **SELECT** se selectează tabul *Worksheet* sau se apasă butonul *SQL* din panoul *Query Result*:

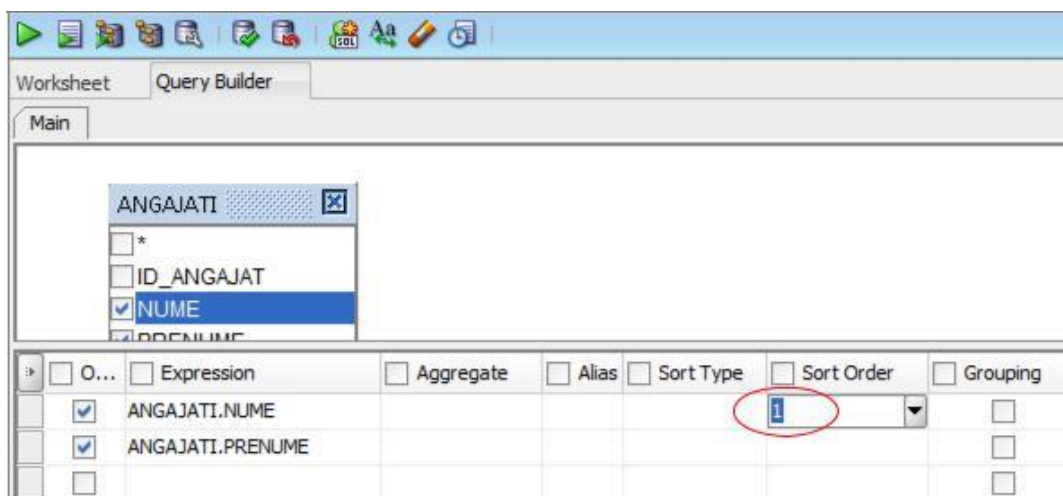


Comanda afișată poate fi copiată folosind butonul *Copy...*

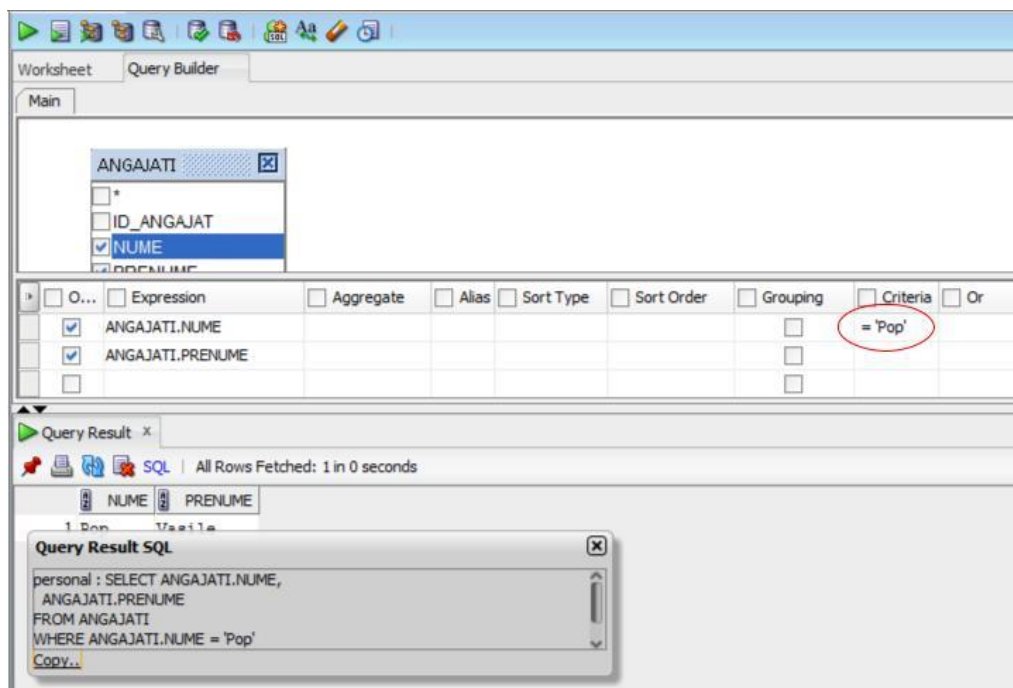
- **SELECT * FROM angajati**

Dacă în locul indicării câmpurilor se selectează « * » mulțimea de selecție va conține toate câmpurile tabelului.

- **SELECT Nume, Prenume FROM angajati ORDER BY Nume**



După indicarea câmpurilor folosite la sortare se poate folosi *Sort Type* pentru a modifica sensul acestora (*ASCENDING* sau *DESCENDING*). În cazul în care se folosesc mai multe câmpuri, acestea sunt separate prin virgulă.



Folosirea unor criterii de selecție (*Criteria*) va genera o comandă SQL în care va fi prezentă și clauza *WHERE*.

- **SELECT** *nume, prenume* **FROM** *angajati* **WHERE** *nume* **LIKE** 'Po%'

În clauza *WHERE* a comenzii *SELECT* se poate folosi caracterul generic « % », cu sensul de « orice șir de caractere ». Comanda precedentă va permite selectarea liniilor în care *nume* = *Pop* sau *nume*=*Popescu*. Clauza *WHERE prenume LIKE '%a'* de exemplu permite selectarea tuturor liniilor în care prenumele se termină în "a".

Pentru a introduce criterii complexe, ca în exemplul anterior sau în exemplele următoare, se va selecta tabul *Worksheet* și comanda va fi tastată în fereastra afișată.

La scrierea condiției se pot folosi operatorii logici *AND*, *OR* sau *NOT* precum și operatorii relaționali:

=	Egal
>	Mai mare
<	Mai mic
>=	Mai mare sau egal
<=	Mai mic sau egal
<> sau !=	Diferit de
LIKE	*Vezi nota

Exemple suplimentare:

```
SELECT * FROM angajati WHERE nume = 'Popescu'  
                AND prenume='Mihai'  
SELECT * FROM angajati WHERE nume = 'Popescu' OR  
nume='Mihai'  
SELECT * FROM angajati WHERE nume = 'Popescu' AND prenume  
                IN ('Ioan','Vasile','Grigore')
```

Clauza **IN** specifică o mulțime căreia trebuie să-i aparțină câmpul specificat (*prenume*).

```
SELECT * FROM angajati WHERE nume = 'Popescu' AND  
data_nasterii BETWEEN '12-MAY-1968' AND '29-MAY-  
1980'
```

Clauza **BETWEEN** permite definirea unui interval căruia trebuie să-i aparțină câmpul specificat (*data_nasterii*).

Clauza **DISTINCT** permite realizarea unei mulțimi de selecție care conține înregistrări distincte, care diferă prin cel puțin o valoare a unui câmp.

De exemplu:

```
SELECT DISTINCT Localitate FROM angajati
```

Efect:

Ultima frază va provoca afișarea numelor localităților în care domiciliază angajații. Fără clauza *DISTINCT*, numele orașului Cluj-Napoca de exemplu ar fi fost probabil repetat de mai multe ori.

Clauza **GROUP BY** permite gruparea înregistrărilor după valoarea unui câmp.

Exemplul 1:

```
SELECT Localitate, COUNT (*) as Nr_Cititori FROM Cititor GROUP BY Localitate
```

În cazul în care se realizează o grupare, fiecare dintre liniile mulțimii de selecție se referă la un grup de înregistrări și nu la înregistrări simple. În exemplul precedent, *GROUP BY Localitate* precizează câmpul după care se realizează gruparea. Înafara valorii câmpului după care se face gruparea, în astfel de situații liniile mulțimii de selecție pot conține rezultatul aplicării unor funcții matematice asupra articolelor care formează grupul (suma valorilor dintr-un câmp, media valorilor, valoarea maximă sau minimă, numărul de articole care formează grupul etc). Pentru câmpurile care vor conține rezultatul aplicării unor funcții se recomandă folosirea clausei *AS nume* pentru atribuirea unui nume relevant coloanei respective. În exemplul dat s-a afișat rezultatul funcției *COUNT (*)*, care numără înregistrările din grup. Funcțiile care pot fi apelate pentru câmpuri numerice sunt prezentate în tabelul de mai jos.

MIN	<i>returnează cea mai mică valoare dintr-o coloană (câmp numeric)</i>
MAX	<i>returnează cea mai mare valoare dintr-o coloană (câmp numeric)</i>
SUM	<i>returnează suma valorilor numerice dintr-o coloană (câmp numeric)</i>
AVG	<i>returnează media valorilor dintr-o coloană (câmp numeric)</i>
COUNT	<i>returnează numărul de valori dintr-o coloană</i>
COUNT DISTINCT	<i>returnează numărul de valori distincte dintr-o coloană</i>

Exemplul 2:

```
SELECT Sectie, MAX (salar) as Sal_Max FROM Angajati
```

```
GROUP BY Sectie
```

Se pot însă aplica aceleași funcții și întregului fișier, fără gruparea articolelor, caz în care mulțimea de selecție va conține o singură înregistrare:

```
SELECT AVG (salar) FROM Angajati
```

```
SELECT AVG (salar) FROM Angajati WHERE functie='Zidar'
```

```
SELECT COUNT (*) FROM Cititor
```

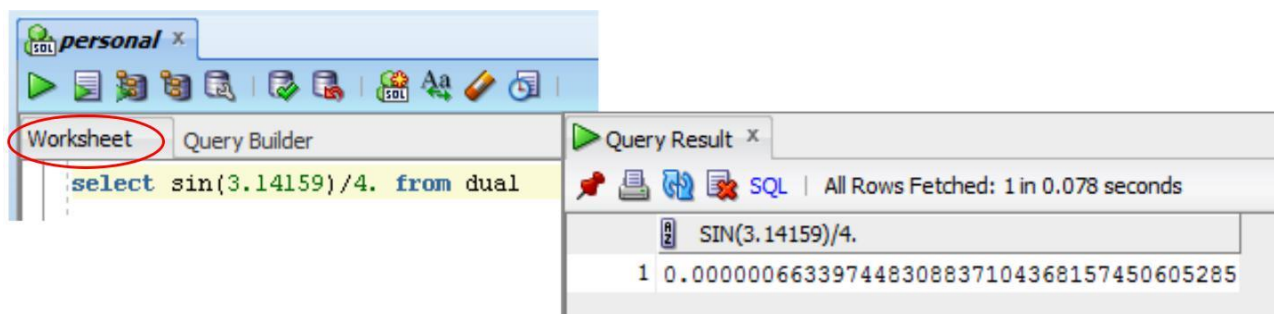
Clauza **HAVING** servește la precizarea unui filtru care se aplică grupurilor de articole, dacă este prezentă clauza **GROUP BY**.

Exemplu:

```
SELECT Sectie, AVG (Salar) FROM Angajati GROUP BY Sectie HAVING  
AVG (Salar) > 500
```

Dacă în fraza **SELECT** lipsește clauza **GROUP BY**, folosirea clauzei **HAVING** nu se justifică, ea având același efect ca și clauza **WHERE**.

O situație aparte prezintă comenzile **SELECT** care nu realizează o mulțime de selecție ci un **calcul matematic**. În acest caz *from nume_tabel* va fi înlocuit prin *from dual*, ca în exemplul următor:



Cuvântul rezervat *dual* astfel folosit permite respectarea sintaxei comenzii **SELECT**.

3. Desfășurarea lucrării

1. Se vor executa exemplele prezentate în lucrare, utilizând bazele de date realizate în laboratoarele anterioare.
2. Se va realiza o bază de date proprie pe care se vor experimenta opțiunile **SELECT**.

4. Temă

1. Afișați structura tabelului `scott.dept` și conținutul lui.

2. Afișați structura tabelului scott.emp. Creați o interogare care să afișeze numele, funcția, data angajării și numărul angajatului pentru fiecare angajat, numărul angajatului apărând pe prima poziție. Salvați instrucțiunea într-un fișier p2.sql.
3. Rulați interogarea din fișierul p2.sql
4. Afișați funcțiile (coloana job) din tabelul scott.emp, eliminând duplicatele.
5. Încărcați fișierul p2.sql în buffer-ul SQL. Modificați instrucțiunea SQL din buffer astfel încât să redenumiți numele coloanelor cu Emp#, Employee, Job, Hire Date și apoi rulați interogarea.
6. Creați o interogare care să afișeze, pe o singură coloană, numele fiecărui angajat concatenat cu funcția sa, separate de virgulă și spațiu. Denumiți coloana Employee and Title.
7. Afișați datele din tabelul scott.emp, concatenând toate coloanele. Separați coloanele cu o virgulă și denumiți coloana THE_OUTPUT.

Rezultatul interogării este de forma:

THE_OUTPUT

7369, SMITH, CLERK, 7902, 17-DEC-80, 800, , 20
7499, ALLEN, SALESMAN, 7698, 20-FEB-81, 1600, 300, 30
7521, WARD, SALESMAN, 7698, 22-FEB-81, 1250, 500, 30

8. Afișați numele și salariul angajaților din tabelul scott.emp care câștigă mai mult de \$2850. Salvați instrucțiunea SQL în fișierul p8.sql și apoi rulați-l.
9. Modificați p8.sql astfel încât să afișați numele și salariul tuturor angajaților ale căror salarii nu intră în intervalul \$1500 - \$2850. Salvați instrucțiunea în fișierul p9.sql și apoi rulați din nou interogarea.
10. Afișați numele și numerele de departament ale angajaților care lucrează în departamentele 10, respectiv 30, ordonați alfabetic după nume.
11. Modificați fișierul p9.sql și listați numele și salariul angajaților care câștigă mai mult de \$1500 și lucrează în departamentul 10 sau 30. Redenumiți coloanele din rezultat Angajat și Salar Lunar. Salvați modificările în fișierul p11.sql și apoi rulați-l.
12. Afișați numele și funcția pentru angajații care nu au manager.
13. Afișați numele, salariul și comisionul pentru toți angajații care au comision. Sortați datele în ordine descendentă după salariu și comision.
14. Afișați numele angajaților care conțin 2 caractere 'L' consecutive în numele lor și îndeplinesc următoarea condiție: lucrează în departamentul 30 sau au manager cu marca 7782.
15. Modificați p11.sql și afișați numele, salariul și comisionul pentru toți angajații care au comisionul mai mare decât salariul mărit cu 10%. Salvați modificările în fișierul p15.sql și apoi rulați-l.
16. Afișați numele, funcția și salariul angajaților ce au funcția Clerk sau Analyst și al căror salariu nu este de \$1000, \$3000 sau \$5000.
17. Afișați numele, funcția și data angajării persoanelor angajate între 10 februarie 1981 și 1 mai 1981. Ordonați înregistrările returnate de interogare crescător după data angajării.